MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Maxim Ormanji FAF-242

Report

Laboratory work No.1-2

Operating System

Checked by:

Reițman Petricia, *university assistant*

DISA, FCIM, UTM

Chișinău – 2026

Before you start

I created the working folder `~/os-lab1` and checked who I am for the OS, which kernel is running and how long the system has been up.

```
vboxuser@Ubuntu:~$ cd os-lab1
vboxuser@Ubuntu:~/os-lab1$ whoami
vboxuser
vboxuser@Ubuntu:~/os-lab1$ uname -a
Linux Ubuntu 7.0.0-34-generic #34-Ubuntu SMP PREEMPT_DYNAMIC Wed Sep  2 14:29:37 UTC 2026
x86_64 GNU/Linux
vboxuser@Ubuntu:~/os-lab1$ uptime
 07:30:43 up 26 min,  1 user,  load average: 0.14, 0.08, 0.12
```

The OS sees me as the user `vboxuser`, and the kernel is Linux 7.0.0-34-generic.

Part 1. Files and directories (file management)

Navigating the tree

```
vboxuser@Ubuntu:~/os-lab1$ pwd
/home/vboxuser/os-lab1
vboxuser@Ubuntu:~/os-lab1$ ls -la /
...
drwxr-xr-x 141 root root 12288 Sep 29 16:36 etc
drwxr-xr-x   3 root root  4096 Sep 29 16:22 home
...
vboxuser@Ubuntu:~/os-lab1$ ls -la ~
...
drwxrwxr-x  2 vboxuser vboxuser 4096 Sep 30 07:42 os-lab1
...
vboxuser@Ubuntu:~/os-lab1$ cd /etc && ls | head
ModemManager
NetworkManager
PackageKit
...
```

Creating, copying, renaming and removing

```
vboxuser@Ubuntu:~/os-lab1$ cd ~/os-lab1
vboxuser@Ubuntu:~/os-lab1$ mkdir demo && cd demo
vboxuser@Ubuntu:~/os-lab1/demo$ echo "hello operating systems" > note.txt
vboxuser@Ubuntu:~/os-lab1/demo$ cp note.txt copy.txt
vboxuser@Ubuntu:~/os-lab1/demo$ mv copy.txt renamed.txt
vboxuser@Ubuntu:~/os-lab1/demo$ ls -l
total 8
-rw-rw-r-- 1 vboxuser vboxuser 24 Oct  1 11:29 note.txt
-rw-rw-r-- 1 vboxuser vboxuser 24 Oct  1 11:30 renamed.txt
vboxuser@Ubuntu:~/os-lab1/demo$ rm renamed.txt
vboxuser@Ubuntu:~/os-lab1/demo$ ls
note.txt
```

Ownership and permissions

```
vboxuser@Ubuntu:~/os-lab1/demo$ ls -l note.txt
-rw-rw-r-- 1 vboxuser vboxuser 24 Oct  1 11:29 note.txt
vboxuser@Ubuntu:~/os-lab1/demo$ chmod 600 note.txt
vboxuser@Ubuntu:~/os-lab1/demo$ ls -l note.txt
-rw------- 1 vboxuser vboxuser 24 Oct  1 11:29 note.txt
vboxuser@Ubuntu:~/os-lab1/demo$ chmod 644 note.txt
```

Observe

1. **Who owns the files and which group?** The files I created (`note.txt` and `renamed.txt`) are owned by the user `vboxuser`, and the group is also `vboxuser`. The names are the same because on Ubuntu every user gets a personal group with the same name.
2. **What do the ten characters mean after `chmod 600`?** The result was `-rw-------`. The first character `-` means it is a normal file (a directory would have `d`). After it there are three groups of three characters: the owner has `rw-` (can read and write, cannot execute), the group has `---` (nothing allowed) and the others have `---` (nothing allowed). So only I can use this file. The number 600 says the same: 6 = 4 (read) + 2 (write), and the two zeros mean no rights.
3. **Two directories under `/`:**
   - `/etc` holds the configuration files of the system and the programs.
   - `/home` holds the personal folders of the users (mine is `/home/vboxuser`).

Part 2. Processes (process management)

Listing the processes

```
vboxuser@Ubuntu:~/os-lab1$ ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.3  0.4  25284 16688 ?        Ss   13:02   0:01 /usr/lib/systemd/systemd --
switched-root --system --deserialize=51
...
vboxuser@Ubuntu:~/os-lab1$ ps aux | wc -l
232
vboxuser@Ubuntu:~/os-lab1$ top
top - 13:12:36 up 10 min,  1 user,  load average: 0.14, 0.33, 0.25
Tasks: 229 total,   1 running, 228 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.7 us,  8.8 sy,  0.0 ni, 84.9 id,  0.0 wa,  0.0 hi,  0.6 si,  0.0 st
MiB Mem :   3398.8 total,    354.2 free,   2068.1 used,   1144.0 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   1330.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2821 vboxuser  20   0 4035984 327048  97400 S  21.0   9.4   0:31.79 gnome-shell
   5018 vboxuser  20   0 1807516 373112 131124 S  11.3  10.7   0:08.64 ptyxis
   4788 vboxuser  20   0   15.1g 428576 119272 S   6.3  12.3   0:26.20 Isolated Web Co
   3857 vboxuser  20   0   11.6g 538168 188180 S   5.7  15.5   0:26.16 firefox
...
```

A background process

```
vboxuser@Ubuntu:~/os-lab1$ sleep 300 &
[1] 5981
vboxuser@Ubuntu:~/os-lab1$ jobs
[1]+  Running                 sleep 300 &
vboxuser@Ubuntu:~/os-lab1$ ps -ef | grep sleep
vboxuser    5981    5057  0 13:16 pts/0    00:00:00 sleep 300
vboxuser    6006    5057  0 13:17 pts/0    00:00:00 grep --color=auto sleep
vboxuser@Ubuntu:~/os-lab1$ kill 5981
[1]+  Terminated              sleep 300
```

The /proc pseudo-filesystem

```
vboxuser@Ubuntu:~/os-lab1$ sleep 300 &
[1] 6453
vboxuser@Ubuntu:~/os-lab1$ PID=$!
vboxuser@Ubuntu:~/os-lab1$ ls /proc/$PID/
...
vboxuser@Ubuntu:~/os-lab1$ cat /proc/$PID/status | head -20
Name:   sleep
...
State:  S (sleeping)
...
VmSize:    16112 kB
...
vboxuser@Ubuntu:~/os-lab1$ kill $PID
[1]+  Terminated              sleep 300
```

Observe

1. **PID of process number 1:** the PID is 1 and the process is `systemd` (the first line of `ps aux` shows `/usr/lib/systemd/systemd`). I looked it up: it is the first process that the kernel starts when the computer boots, and it starts the other services and programs.
2. **How many processes on the idle VM?** `ps aux | wc -l` printed 232, and one line of that is the header, so about 231 processes. `top` showed 229 tasks (1 running, 228 sleeping). So roughly 230 processes. The numbers are a little different because processes start and stop all the time. Many of them have names in square brackets, like `[kworker/...]`, and these are kernel threads, not normal programs.
3. **The `State:` line of the sleeping process:** it says `S (sleeping)`. The `sleep 300` process is just waiting for its timer and does not use the CPU, the OS wakes it up when the time is over. On my idle VM almost all processes are sleeping like this (228 of 229 in `top`).

Part 3. Memory (memory management)

```
vboxuser@Ubuntu:~/os-lab1$ free -h
               total        used        free      shared  buff/cache   available
```

```
Mem:           3.3Gi       2.3Gi       175Mi        62Mi       1.0Gi       1.0Gi
Swap:             0B          0B          0B
vboxuser@Ubuntu:~/os-lab1$ cat /proc/meminfo | head -6
MemTotal:        3480360 kB
MemFree:          179168 kB
MemAvailable:    1059544 kB
...
vboxuser@Ubuntu:~/os-lab1$ sleep 300 & PID=$!
[1] 6829
vboxuser@Ubuntu:~/os-lab1$ grep VmRSS /proc/$PID/status
VmRSS:      7684 kB
```

Observe

1. **Total RAM and free RAM:** `free -h` shows 3.3 GiB total (`MemTotal` is 3480360 kB) and only 175 MiB free. It looks very low, but Linux uses spare RAM as cache (`buff/cache` is about 1.0 GiB) and gives it back to programs when they need it. So the column `available` (1.0 GiB) is the more honest number of memory that programs can still get.
2. **What is swap and how much is configured?** Swap is a space on the disk that the OS can use like extra memory: when RAM is full, it moves the pages that are not used much to the disk. On my VM swap is `0B`, so no swap is configured.
3. **VmRSS of a bare `sleep`:** `VmRSS` is 7684 kB, about 7.5 MB. It surprised me a bit, because `sleep` does nothing, but I think it still needs its code and the shared libraries (like libc) loaded in RAM. Also, in the `/proc` block of Part 2 the `VmSize` of a similar `sleep` process was 16112 kB, so the virtual size is bigger than the RAM it really uses.

Part 4. Devices and storage (I/O management)

```
vboxuser@Ubuntu:~/os-lab1$ df -h
Filesystem      Size  Used Avail Use% Mounted on
...
/dev/sda2        30G  6.8G   22G  25% /
...
/dev/sr0         51M   51M     0 100% /run/media/vboxuser/VBox_GAs_7.2.20
vboxuser@Ubuntu:~/os-lab1$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
...
loop4    7:4    0 260.3M  1 loop /snap/firefox/8763
...
sda      8:0    0    30G  0 disk
|-sda1   8:1    0     1M  0 part
`-sda2   8:2    0    30G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/vboxuser/VBox_GAs_7.2.20
vboxuser@Ubuntu:~/os-lab1$ du -sh ~/os-lab1
28K     /home/vboxuser/os-lab1
vboxuser@Ubuntu:~/os-lab1$ ls -l /dev | head
...
lrwxrwxrwx+ 1 root     root          3 Oct  2 13:00 cdrom -> sr0
```

```
...
vboxuser@Ubuntu:~/os-lab1$ mount | head
...
/dev/sda2 on / type ext4 (rw,relatime)
...
```

Observe

1. **Device of the root filesystem `/`:** it is mounted on `/dev/sda2` (the line `/dev/sda2 30G 6.8G 22G 25% /` in `df -h`). `mount` shows that its type is `ext4`. In `lsblk` it is the partition `sda2` of the 30 GB disk `sda`.
2. **One entry from `/dev`:** `/dev/cdrom`, which is a link to `sr0`. It stands for the CD/DVD drive. In my VM it is a virtual drive with the VirtualBox Guest Additions disc (it is also visible in `df -h` as `/dev/sr0`).
3. **"Everything is a file":** the OS shows disks, terminals and even information about processes in the same way as normal files, with paths and read/write operations. For example, I read `/proc/6453/status` like a text file, and my disk appears as `/dev/sda2`.

Conclusion

The OS manages four resources: files, processes, memory and devices. I saw the files with `ls -l` (owner, group and permissions of `note.txt`), the processes with `ps aux` (about 230 processes, and `systemd` is PID 1) and the memory with `free -h` (3.3 GiB of RAM and no swap). I saw the devices with `lsblk`, which showed the disk `sda` that my root filesystem `/` is mounted on.

Part 1. Build it and boot it

I cloned the xv6 repository, went into the folder and ran `make qemu`. The kernel booted and showed the `$` prompt.

```
vboxuser@Ubuntu:~/os-lab2$ git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
vboxuser@Ubuntu:~$ cd ~/xv6-riscv
vboxuser@Ubuntu:~/xv6-riscv$ make qemu

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

Part 2. Use xv6 as the Unix it is

I ran the built-in programs: `ls`, `cat README`, `echo hello xv6`, `ls | grep c`, `wc README` (the output of `cat README` is shortened).

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
console        3 23 0
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.

...
$ echo hello xv6
```

```
hello xv6
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
$ wc README
48 336 2441 README
```

Observe (Part 2)

1. **List three programs xv6 ships with.**
   `ls`, `cat` and `echo`. The `ls` output also shows `grep`, `wc`, `mkdir`, `rm`, `kill`, `ln` and others.
2. **You just used a pipe (`|`) inside xv6. Which two OS features must exist for a pipe to work?**
   Processes and communication between processes. In `ls | grep c` two programs run (`ls` and `grep`), so the OS must be able to create processes. The output of `ls` has to reach the input of `grep`, so there must be a communication channel (a pipe) provided by the kernel.
3. **In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?**
   The xv6 shell is much simpler than the Linux shell, but it does the basics: it runs programs and supports pipes.

Part 3. Read the source

I quit QEMU and read `user/ls.c` and `user/cat.c`, looked at the list of system calls in `user/user.h`, and searched `kernel/sysfile.c` for `sys_read` and `sys_write`. Only the parts needed for the answers are shown below.

Beginning of `user/cat.c`:

```
vboxuser@Ubuntu:~/xv6-riscv$ sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
```

```
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
```

List of system calls in `user/user.h`:

```
vboxuser@Ubuntu:~/xv6-riscv$ cat user/user.h
// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);
...
```

Where `sys_read` and `sys_write` are implemented:

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
```

Observe (Part 3)

1. **Which system calls does `user/cat.c` use, and what does each one ask the kernel to do?**
   - `read()` asks the kernel to read data from a file (by its descriptor) into a buffer.
   - `write()` asks the kernel to write data to the output (descriptor 1, the screen).
   - `exit()` asks the kernel to end the program.
   - `open()` and `close()` (in the second half of the file, in `main`) ask the kernel to open a

file and later close it.

2. **In which kernel file, and on which line, is `sys_read` implemented?**
   In `kernel/sysfile.c`, line 69 (`sys_write` is on line 83).
3. **In one sentence: what is the difference between the code in `kernel/` and the code in `user/`?**
   `kernel/` contains the operating system itself, which manages memory, files and processes, while `user/` contains ordinary programs (`ls`, `cat`, the shell) that ask the kernel for services through system calls.

Part 4. Write your first xv6 program

I created `user/sleep.c`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

Then I added `$U/_sleep` to the `UPROGS` list in the `Makefile` (beginning of the list):

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_sleep\
...
```

After rebuilding with `make qemu`, I tested the program inside xv6:

```
vboxuser@Ubuntu:~/xv6-riscv$ make qemu
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

Note. In my version of xv6 the system call that puts a process to sleep is called `pause` (see `int pause(int);` in `user/user.h`), not `sleep` as in the lab text. That is why `sleep.c` calls `pause(atoi(argv[1]))`. The program itself is still named `sleep`.

Result: `sleep 10` pauses and then returns to the prompt, and `sleep` without an argument prints `usage: sleep <ticks>`.

Conclusion

I built and booted xv6, used it as a Unix system, saw how the kernel code (`kernel/`) is separated from user programs (`user/`), and wrote the `sleep` program, which makes a system call into a kernel that I built myself.